

DL Homework 2

Diogo Sousa

107004

diogomppsousa@tecnico.ulisboa.pt

Florian Huhnd

1117194

florian.huhnd@tecnico.ulisboa.pt

Diogo Pica

105855

diogo.pica@tecnico.ulisboa.pt

Abstract

This report deals with the two exercises of Deep Learning Homework 2: blood cell image classification (BloodMNIST) and RNA-protein binding affinity prediction (RNA-compete/RBFOX1). For Question 1, we compare CNN training with logits vs. an explicit Softmax layer and analyze the impact of Max-Pooling. For Question 2, we evaluate a 1D CNN and a bi-directional GRU using masked MSE loss and Spearman correlation, and we test a multi-head self-attention extension. The last part of Question 2 briefly describes how to extend the setup to multi-protein binding prediction.

Diogo Pica answered Question 1. Florian Huhnd and Diogo Sousa worked on Question 2, with Florian doing the CNN and Question 2.3 and Diogo doing RNN and 2.2.

1 Question 1

1.1 CNN network comparison between softmax and non softmax

A convolution neural network (CNN) was trained on the BloodMNIST dataset, consisting of 17902 RGB blood cells images from 8 different classes, using the previously defined 7:1:2 train, validation, test split. Images are center-cropped and resized to $3 \times 28 \times 28$. The network architecture included 3 convolutional layers with 32, 64, and 128 channels, each followed by ReLU activation and two fully connected layers with 256 hidden units and 8 output classes. Two configurations were evaluated: one with a Softmax layer at the output and one producing raw logits. Both models were trained for 200 epochs using Adam (with a learning rate of 0.001), batch size 64, and Cross-Entropy loss. [Campos et al., 2025].

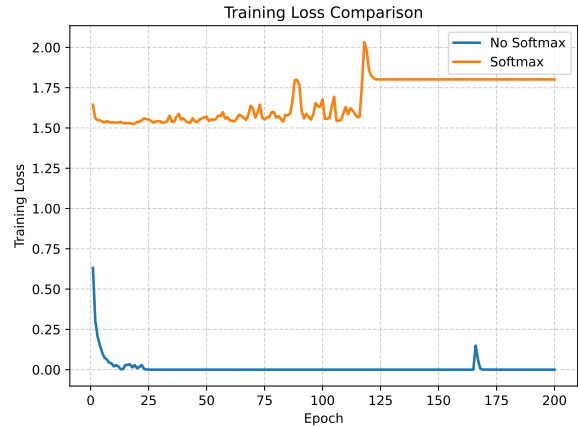


Figure 1: Training loss for CNN with both Softmax and without Softmax.

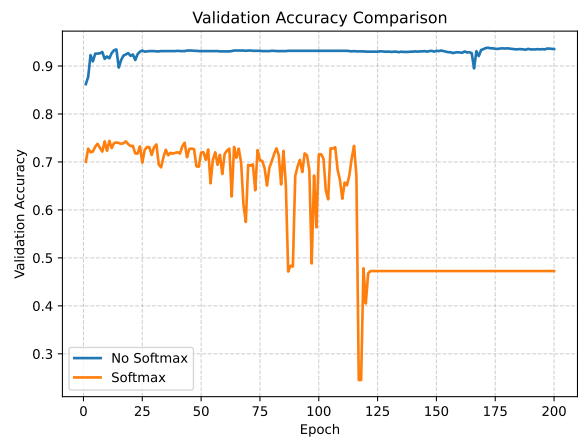


Figure 2: Validation accuracy for CNN with both Softmax and without Softmax.

The training behavior highlights a pronounced performance difference between the two configurations. When a Softmax layer is applied at the output, the training loss, as demonstrated in figure 1, decreases slowly and remains high, while the validation accuracy, presented in figure 2 stays below 75% suggesting sub-optimal convergence and limited generalization.

This behavior is explained by the fact that the

CrossEntropyLoss function internally applies a Log-Softmax operation. Explicitly adding a Softmax layer leads to redundant normalization and poorly scaled gradients, degrading learning performance. Based on the validation results, the model without Softmax was selected and achieved a test accuracy of 92.40 %. [PyTorch Team]

Overall, the proposed CNN without Softmax is well-suited for BloodMNIST classification. Despite its simplicity and lack of pooling layers, it achieves high accuracy on a challenging multi-class medical dataset, making it a strong baseline for further architectural improvements.

1.2 CNN network comparison between softmax and non softmax with an addition of 3 Maxpool layers

To evaluate the impact of spatial downsampling, 3 MaxPool2d(2) layers were added after each convolution block and the previous experimental protocol described in question 1 was followed. 2 configurations were evaluated Logits + MaxPool and Softmax + MaxPool. [Campos et al., 2025].

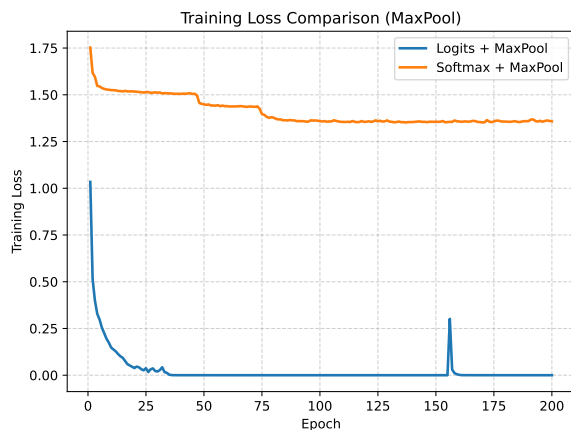


Figure 3: Training loss for CNN using Maxpool with both Softmax and without Softmax.

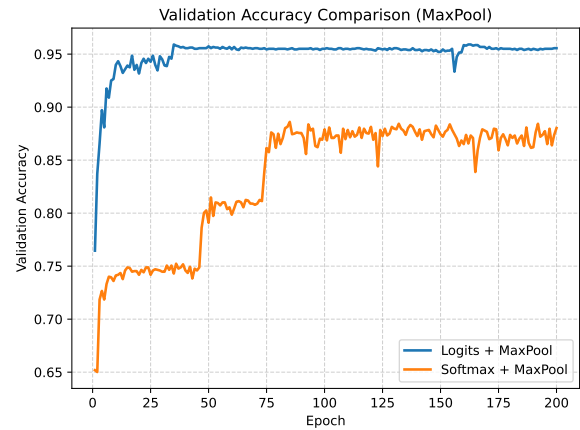


Figure 4: Validation accuracy for CNN using Maxpool with both Softmax and without Softmax.

The introduction of MaxPooling leads to a noticeable improvement in classification performance for both configurations. The Logits + MaxPool model achieves the highest accuracy, reaching a test accuracy of 95.00%, while the Softmax + MaxPool model achieves 87.69%. Validation curves show that the logits-based model converges faster and stabilizes at a higher accuracy level compared to the Softmax variant, showed in figure 4. The corresponding training loss curves further confirm more stable and effective optimization when raw logits are used, represented in figure 3, despite showing an improvement comparably with the previous question without Maxpool.

Compared to the models without MaxPooling presented in question 1, the accuracy improvement indicates that MaxPooling enhances the network's ability to capture more robust and spatially invariant features, which is particularly beneficial given the small input resolution of the images.

In terms of computational efficiency, the inclusion of MaxPooling does not introduce a significant increase in training time. The Logits + MaxPool model required 820 seconds, while the Softmax + MaxPool model required 816 seconds to complete 200 epochs.

Overall, MaxPooling improves model effectiveness without compromising efficiency. As observed in Question 1, the logits-only configuration outperforms the Softmax-based model due to better interaction with the Cross-Entropy loss. When combined with MaxPooling this configuration yields the best overall performance achieving the highest accuracy and stable convergence.

2 Question 2

2.1 Deep Neural Network Architectures

We address RBFOX1 RNA-protein binding affinity prediction as a regression task on one-hot encoded RNA sequences, where model performance is evaluated primarily via Spearman rank correlation (rather than absolute error), according to the exercise [Campos et al., 2025].

We selected architectures previously covered in class (CNN/RNN) because they provide inductive biases suitable for sequential biological data: 1D CNNs capture local motif patterns with shared parameters across positions (useful for sequence motifs, see L7p16), while sequence models like RNNs can capture longer-range dependencies and global interactions (L8p28). [Martins et al., 2025a,b]

2.1.1 First model: CNN

Model. Our first model is a 1D convolutional neural network (CNN). CNNs are good at detecting short local patterns because filters slide across the sequence with shared parameters, which is useful when motifs can appear at different positions. Pooling progressively reduces sequence length and keeps the strongest activations, which helps to find the most “motif-like” evidence (L7p19 ff.). [Martins et al., 2025a]

Hyperparameter search. We used a small grid search running over a combination of the following hyperparameters using the validation split only: (i) learning rate $\in \{0.001, 0.0005\}$, (ii) dropout $\in \{0.2, 0.3\}$, (iii) kernel size $\in \{5, 7\}$, (iv) number of channels $\in \{(32, 64, 128), (64, 128, 256)\}$.

The best CNN configuration (by validation Spearman) was: learning rate 0.001, dropout 0.2, kernel size 5, channels (64, 128, 256), with best epoch 37. It achieved a validation Spearman of 0.644 and a **test Spearman of 0.642** (test loss 0.396).

Learning curves. Figure 5 reports the train and validation loss across epochs for the selected CNN run. We observe a rapid decrease in loss in early epochs followed by a slower improvement/plateau, and we choose the final checkpoint based on the best validation Spearman rather than the final epoch. This prevents bias from late-epoch overfitting.

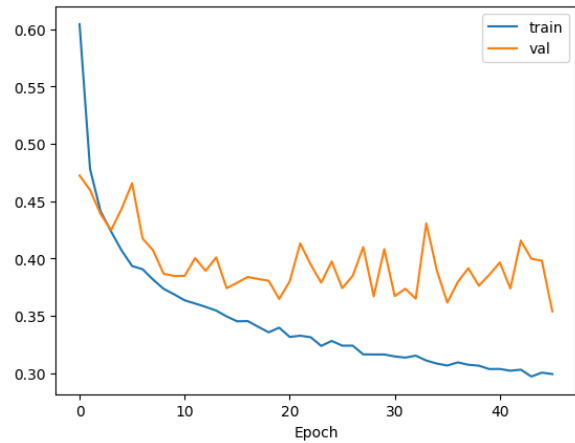


Figure 5: Training and validation loss for the selected CNN configuration.

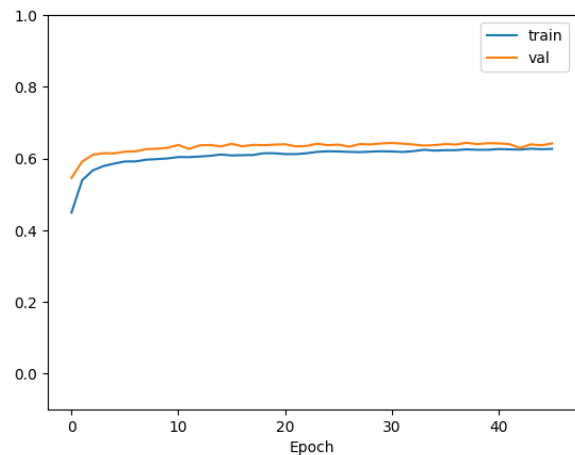


Figure 6: Training and validation Spearman for the selected CNN configuration.

2.1.2 Second model: RNN (GRU)

Describe the second model, the hyperparameters you chose to vary, and the hyperparameters of the selected checkpoint. Provide requested plots.

Model. Recurrent Neural Networks (RNN) are commonly used for sequential data. They have a memory in the form of the *hidden state*, which is keeps information about previous inputs. The recurring units in RNN's are, however, susceptible to vanishing/exploding gradients, for that reason, alternative network architectures were created. We opted to implement a Gated Recurrent Unit (GRU) network, because these generally outperform Long Short-Term Memory (LSTM) networks on low-complexity sequences [Cahuantzi et al., 2023] and are more computationally efficient. This architecture uses gating mechanisms, which allow the GRU

to selectively remember or forget information from previous time steps [Outemzabet et al., 2024].

Hyperparameter search. We used a grid search running over the following hyperparameters: (i) learning rate $\in \{0.001, 0.0005\}$, (ii) dropout $\in \{0.2, 0.3\}$, (iii) number of layers $\in \{1, 2\}$, (iv) hidden dimensions $\in \{128, 256\}$, (v) Bi-directional $\in \{True, False\}$

The best configuration was: 0.0005 learning rate, 0.2 dropout, 1 layer, 256 hidden dimensions. It achieved a validation Spearman of 0.665 and a **test Spearman of 0.659** (test loss 0.334). Since the sequences are short, a single GRU layer was optimal, while the higher hidden dimensions value allowed for the encoding of more complex features. As expected, Bi-directional models are stronger, because both left and right context is relevant in RNA sequences.

Learning curves. The training was stopped early at epoch 20 when the patience value was hit. Stopping avoids the overfitting that is noticeable in the last epochs, when the training loss keeps dropping but the validation is at a plateau.

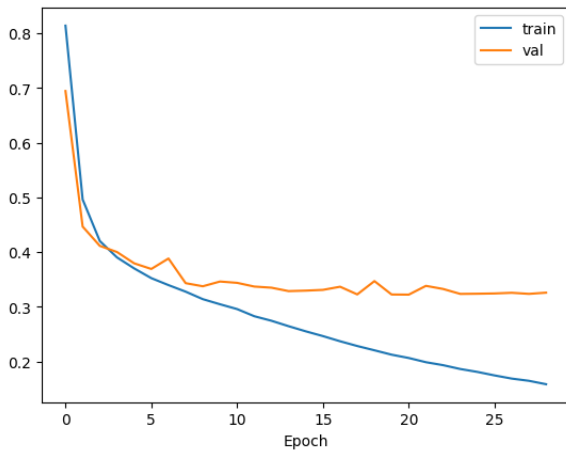


Figure 7: Training and validation loss for the selected RNN configuration.

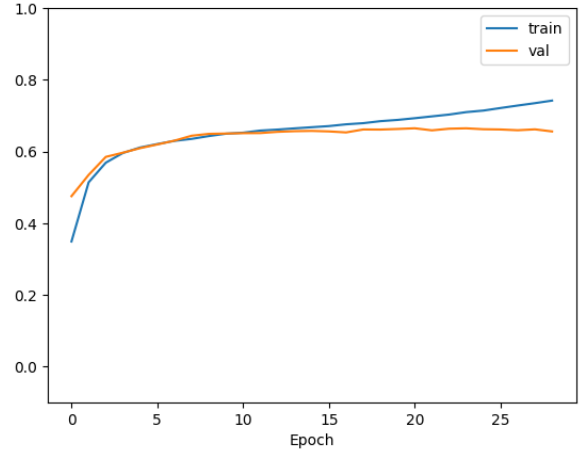


Figure 8: Training and validation Spearman for the selected RNN configuration.

2.1.3 Comparative results.

The RNN model marginally outperformed the CNN model. It was also able to reach a visible plateau, while the validation loss graph keeps varying over its last epochs before early stopping. 7 2

Model	Val (best)	Test
CNN	0.644	0.642
RNN	0.665	0.659

Table 1: Comparison of model performance (Spearman) for RBFOX1.

We expected both models to reach similar scores because, while having different strengths, they are both suited for the problem. The CNN is suited to detect local translation-invariant motifs like *UG-CAUG*. The RNN has the advantage of better capturing context across the full RNA sequence.

2.2 Attention

We extended our best-performing architecture (the Bi-directional GRU) by incorporating a Multi-Head Self-Attention mechanism. We chose 4 heads to allow the model to capture multiple motifs simultaneously.

2.2.1 Results.

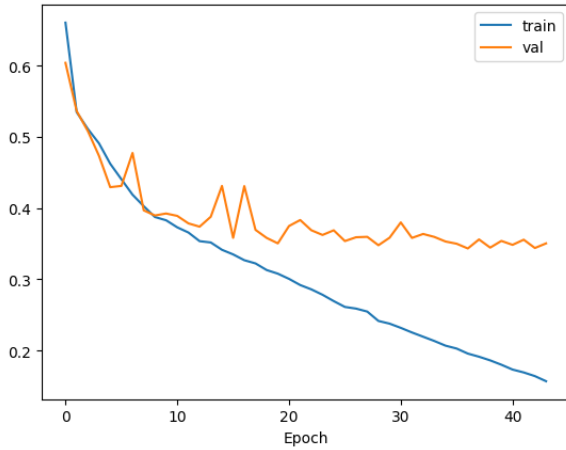


Figure 9: Training and validation loss for the MHSA RNN configuration.

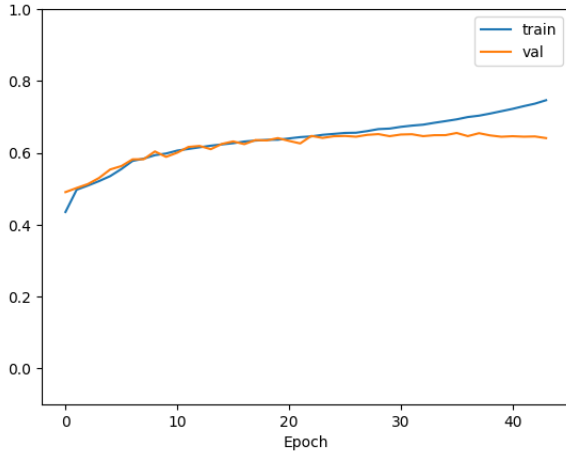


Figure 10: Training and validation Spearman for the MHSA RNN configuration.

Model	Val (best)	Test
RNN	0.665	0.659
RNN (MHSA)	0.655	0.647

Table 2: Comparison of model performance (Spearman) for RBFOX1.

We initially expected that the attention mechanism would improve performance by allowing the model to focus on the high-affinity motif regions.

In reality, there was a slight drop in test performance 2. We think this can be explained by the short length of the RNA sequences (38-41). The Bi-directional GRU is already capable of propagating information across such a short distance, without an additional attention mechanism. The added

complexity might be introducing noise or causing overfitting instead of capturing new information.

2.3 Multiple RNA-Binding Proteins

2.3.1 Proposed modifications

Multi-protein affinity prediction changes the input from only RNA to (RNA, protein), so the dataset must include protein identity/features and an affinity label for each RNA-protein pair.

The architecture of the model needs two parts: an RNA encoder, which can be the same CNN as used before, and a protein encoder (e.g., an embedding for protein ID or features from protein sequence), whose outputs are combined (e.g., concatenation + MLP) to predict the affinity for that specific protein.

Training can still use masked MSE and evaluation with Spearman, but Spearman should be computed per protein and averaged to avoid domination by proteins with more samples, and validation must be used for tuning while test is reserved for final reporting.

2.3.2 Anticipated challenges and benefits

One benefit of training a single model on multiple RNA-binding proteins is data efficiency: the model can learn general RNA pattern features from all proteins and reuse them, which can help proteins with fewer labeled examples.

One challenge is protein heterogeneity/imbalance: different RNA-binding proteins can have very different binding rules (and some may have many more examples), so one shared model can end up fitting the “dominant” proteins well while performing worse on others unless training and evaluation are carefully balanced.

3 Collaboration with other teams and use of AI tools

We did not collaborate with other groups.

In 1.1 and 1.2 AI was used to help implementation of CNN and code debugging. We took concepts from practical 5 and practical 7 and tried to implement them by hand using AI only only when problems occurred.

AI was used in the same way to help with the implementation of the CNN and RNN in 2.1. Additionally, we let AI add the option to run the whole grid search after implementing the single runs first (to quickly check if the code is working).

In 2.3 we were able to find the needed changes to the architecture on our own, but AI helped finding

different challenges to watch out for. We made sure to truly understand every comment given by AI.

4 Conclusions

On BloodMNIST, removing the explicit Softmax improves optimization with CrossEntropyLoss and yields higher accuracy, and adding MaxPooling further improves performance without increasing runtime. On RNAcompete (RBFOX1), both CNN and bi-directional GRU perform similarly, with the GRU achieving slightly higher Spearman correlation. Adding multi-head self-attention to the GRU did not improve results, likely because the sequences are short and the base model already captures global context. Multi-protein affinity prediction can be achieved by changing the architecture of the code.

References

- Roberto Cahuantzi, Xinye Chen, and Stefan Güttel. 2023. *A Comparison of LSTM and GRU Networks for Learning Symbolic Sequences*, page 771–785. Springer Nature Switzerland.
- Margarida Campos, Guilherme Coimbra, Faria, and Chrysoula Zerva. 2025. Deep learning ist (2025–26) homework 2. Course handout. Instituto Superior Técnico.
- André Martins, Francisco Melo, and Mário Figueiredo. 2025a. Deep learning (ist) lecture 7: Convolutional neural networks. Course slides. Slides 16–24. Parameter tying in 1D sequences, convolutions, pooling for invariance.
- André Martins, Francisco Melo, and Mário Figueiredo. 2025b. Deep learning (ist) lecture 8: Recurrent neural networks. Course slides. Slides 4–5, 28–30. RNNs for sequential models and arbitrary-length sequences.
- Leila Outemzabet, Nicolas Gaud, Aurélie Bertaux, Christophe Nicolle, Stéphane Gerart, and Sébastien Vachenc. 2024. *Harnessing heterogeneous information networks: A systematic literature review*. *Computer Science Review*, 52:100633.
- PyTorch Team. `torch.nn.crossentropyloss` — pytorch documentation. <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>. Accessed: January 2026.